

rtCourse: Advanced Databases

Group: A1

Team Members: Tobiasz Bazan, Jakub Kamiński, Adam Ćwikliński

Assignment: Columnar store

Objective

The objective of this phase is to investigate the impact of Oracle columnar storage and in-memory technologies on workload performance. Based on query plan analysis, suitable tables and columns will be selected for columnar storage optimization. The study will evaluate the influence of these enhancements on both analytical queries and CRUD operations. Additional Oracle features, including In-Memory Expressions and Join Groups, will be prepared and tested as part of the experiments. The phase will also compare different storage and compression configurations to determine their effect on execution time, query plans, and overall workload efficiency.

Tables / columns candidates for columnar store

Table:

TELEMETRY_DATA

Columns:

activity_id

heart_rate_bpm

pace

running_power

Query plan for transaction 2:

Id	Operation	Name	Rows	Bytes	TempSp	Cost (%CPU)	Time
0	SELECT STATEMENT		10	2200		4532 (1)	00:00:01
1	SORT ORDER BY		10	2200		4532 (1)	00:00:01
* 2	VIEW		10	2200		4531 (1)	00:00:01
* 3	WINDOW SORT PUSHED RANK		8	784		4531 (1)	00:00:01
* 4	FILTER						
5	HASH GROUP BY		8	784		4531 (1)	00:00:01
6	VIEW	VW_DAG_0	2896	277K		4530 (1)	00:00:01
7	HASH GROUP BY		2896	285K		4530 (1)	00:00:01
* 8	HASH JOIN OUTER		2896	285K		4529 (1)	00:00:01
* 9	HASH JOIN		673	62589		4522 (1)	00:00:01
* 10	TABLE ACCESS FULL	RUNNERS	673	16152		5 (0)	00:00:01
11	VIEW	VW_GBC_30	1038	71622		4517 (1)	00:00:01
12	HASH GROUP BY		1038	83040		4517 (1)	00:00:01
* 13	HASH JOIN		9335	729K		4516 (1)	00:00:01
14	JOIN FILTER CREATE	:BF0000	9335	446K		113 (1)	00:00:01
* 15	HASH JOIN		9335	446K		113 (1)	00:00:01
* 16	TABLE ACCESS FULL	ROUTES	239	3107		3 (0)	00:00:01
* 17	HASH JOIN		9758	343K		110 (1)	00:00:01
* 18	TABLE ACCESS FULL	SHOES	464	8816		7 (0)	00:00:01
* 19	TABLE ACCESS FULL	ACTIVITIES	25237	418K		103 (1)	00:00:01
20	VIEW		36120	1093K		4403 (1)	00:00:01
21	HASH GROUP BY		36120	458K	24M	4403 (1)	00:00:01
22	JOIN FILTER USE	:BF0000	1080K	13M		2469 (1)	00:00:01
* 23	TABLE ACCESS FULL	TELEMETRY_DATA	1080K	13M		2469 (1)	00:00:01
24	TABLE ACCESS FULL	RUNNER_ACHIEVEMENTS	5000	40000		7 (0)	00:00:01
25	SORT AGGREGATE		1	13			
26	VIEW		1200	15600		4511 (1)	00:00:01
27	HASH GROUP BY		1200	51600		4511 (1)	00:00:01
* 28	HASH JOIN		25237	1059K		4509 (1)	00:00:01
* 29	HASH JOIN		25237	616K		106 (1)	00:00:01
30	TABLE ACCESS FULL	ROUTES	250	2000		3 (0)	00:00:01
* 31	TABLE ACCESS FULL	ACTIVITIES	25237	418K		103 (1)	00:00:01
32	VIEW	VW_GBF_11	36120	634K		4403 (1)	00:00:01
33	SORT GROUP BY		36120	72240		4403 (1)	00:00:01
34	VIEW		36120	72240		4403 (1)	00:00:01
35	HASH GROUP BY		36120	458K	24M	4403 (1)	00:00:01
36	TABLE ACCESS FULL	TELEMETRY_DATA	1080K	13M		2469 (1)	00:00:01
37	SORT AGGREGATE		1	13			
38	VIEW		1200	15600		4125 (1)	00:00:01
39	HASH GROUP BY		1200	37200		4125 (1)	00:00:01
* 40	HASH JOIN		25237	764K		4124 (1)	00:00:01
* 41	TABLE ACCESS FULL	ACTIVITIES	25237	320K		103 (1)	00:00:01
42	VIEW		36120	634K		4021 (1)	00:00:01
43	SORT GROUP BY		36120	317K	20M	4021 (1)	00:00:01
44	TABLE ACCESS FULL	TELEMETRY_DATA	1080K	9500K		2467 (1)	00:00:01

Query plan for transaction 3:

Id	Operation	Name	Rows	Bytes	TempSp	Cost (%CPU)	Time
0	SELECT STATEMENT		2896	622K		13021 (1)	00:00:01
1	SORT ORDER BY		2896	622K	696K	13021 (1)	00:00:01
* 2	VIEW		2896	622K		4836 (1)	00:00:01
* 3	WINDOW SORT PUSHED RANK		2896	350K		4836 (1)	00:00:01
4	HASH GROUP BY		2896	350K		4836 (1)	00:00:01
5	VIEW	VW_DAG_0	2896	350K		4835 (1)	00:00:01
6	HASH GROUP BY		2896	359K		4835 (1)	00:00:01
* 7	HASH JOIN OUTER		2896	359K		4834 (1)	00:00:01
* 8	HASH JOIN		673	80087		4827 (1)	00:00:01
* 9	TABLE ACCESS FULL	RUNNERS	673	16152		5 (0)	00:00:01
10	VIEW	VW_GBC_15	1038	98610		4822 (1)	00:00:01
11	HASH GROUP BY		1038	96534		4822 (1)	00:00:01
* 12	HASH JOIN		9335	847K		4821 (1)	00:00:01
13	JOIN FILTER CREATE	:BF0000	9335	446K		113 (1)	00:00:01
* 14	HASH JOIN		9335	446K		113 (1)	00:00:01
* 15	TABLE ACCESS FULL	ROUTES	239	3107		3 (0)	00:00:01
* 16	HASH JOIN		9758	343K		110 (1)	00:00:01
* 17	TABLE ACCESS FULL	SHOES	464	8816		7 (0)	00:00:01
* 18	TABLE ACCESS FULL	ACTIVITIES	25237	418K		103 (1)	00:00:01
19	VIEW		36120	1552K		4707 (1)	00:00:01
20	HASH GROUP BY		36120	599K	33M	4707 (1)	00:00:01
21	JOIN FILTER USE	:BF0000	1080K	17M		2470 (1)	00:00:01
* 22	TABLE ACCESS FULL	TELEMETRY_DATA	1080K	17M		2470 (1)	00:00:01
23	TABLE ACCESS FULL	RUNNER_ACHIEVEMENTS	5000	40000		7 (0)	00:00:01
24	SORT AGGREGATE		1	13			
25	VIEW		36120	458K		4021 (1)	00:00:01
26	HASH GROUP BY		36120	317K	20M	4021 (1)	00:00:01
27	TABLE ACCESS FULL	TELEMETRY_DATA	1080K	9500K		2467 (1)	00:00:01
28	SORT AGGREGATE		1	13			
29	VIEW		36120	458K		4023 (1)	00:00:01
30	HASH GROUP BY		36120	317K	20M	4023 (1)	00:00:01
31	TABLE ACCESS FULL	TELEMETRY_DATA	1080K	9500K		2470 (1)	00:00:01

Query plan for transaction 6:

Id	Operation	Name	Rows	Bytes	TempSpc	Cost (%CPU)	Time
0	DELETE STATEMENT		164K	3854K		5544 (1)	00:00:01
1	DELETE	TELEMETRY_DATA	164K	3854K		5544 (1)	00:00:01
* 2	HASH JOIN RIGHT SEMI		5495	71435		4895 (1)	00:00:01
3	VIEW	VW_NS0_1					
4	TEMP TABLE TRANSFORMATION						
5	LOAD AS SELECT (CURSOR DURATION MEMORY)	SYS_TEMP_0FD9D660F_16A64F2	36120	599K	33M	4707 (1)	00:00:01
6	HASH GROUP BY		1080K	17M		2470 (1)	00:00:01
7	TABLE ACCESS FULL	TELEMETRY_DATA	5495	413K		137 (2)	00:00:01
* 8	HASH JOIN		5495	203K		111 (1)	00:00:01
* 9	HASH JOIN		121	847		3 (0)	00:00:01
* 10	TABLE ACCESS FULL	ROUTES	11400	345K		108 (1)	00:00:01
* 11	HASH JOIN		542	7588		5 (0)	00:00:01
* 12	TABLE ACCESS FULL	RUNNERS	25237	418K		103 (1)	00:00:01
* 13	TABLE ACCESS FULL	ACTIVITIES	36120	1375K		25 (0)	00:00:01
* 14	VIEW		36120	599K		25 (0)	00:00:01
15	TABLE ACCESS FULL	SYS_TEMP_0FD9D660F_16A64F2	1	13		25 (0)	00:00:01
16	SORT AGGREGATE		36120	458K		25 (0)	00:00:01
17	VIEW		36120	599K		25 (0)	00:00:01
18	TABLE ACCESS FULL	SYS_TEMP_0FD9D660F_16A64F2	1	13		25 (0)	00:00:01
19	SORT AGGREGATE		36120	458K		25 (0)	00:00:01
20	VIEW		36120	599K		25 (0)	00:00:01
21	TABLE ACCESS FULL	SYS_TEMP_0FD9D660F_16A64F2	1080K	11M		647 (1)	00:00:01
22	INDEX FAST FULL SCAN	IDX_TELEMETRY_ACTIVITY_ID					

Analysis:

The query plans presented above indicate that TELEMETRY_DATA is a suitable candidate for Oracle In-Memory Column Store. The plans contain repeated TABLE ACCESS FULL operations over approximately 1,080K rows together with HASH GROUP BY, HASH JOIN and aggregation operations. This suggests that the workload performs analytical processing and repeatedly scans large amounts of telemetry data. Therefore, storing selected TELEMETRY_DATA columns in columnar format is expected to improve scan efficiency and aggregation performance. The most relevant columns are activity_id, heart_rate_bpm, pace and running_power, as they are frequently used in joins, grouping and analytical calculations.

Affected workload operations

Query	Telemetry_Datausage	Description
Q1	Moderate-High	telemetry access + aggregation + partial index support
Q2	Very High	repeated TABLE ACCESS FULL + HASH GROUP BY
Q3	Very High	repeated TABLE ACCESS FULL + HASH GROUP BY + SORT AGGREGATE
Q4	High	TABLE ACCESS FULL + HASH GROUP BY during INSERT selection
Q5	Moderate	telemetry subqueries + TABLE ACCESS FULL
Q6	Very High	DELETE + full telemetry scans + HASH GROUP BY

Analysis:

The workload analysis based on query plans indicates that the impact of Oracle In-Memory Column Store is expected to vary across transactions. The strongest improvements are anticipated for Q2, Q3 and Q6, where TELEMETRY_DATA is repeatedly accessed through full table scans over approximately 1,080K rows together with HASH GROUP BY, SORT AGGREGATE and other analytical operations. Q4 is also expected to benefit, although to a slightly lower degree, because the INSERT transaction performs analytical processing over telemetry data before loading records into ACTIVITIES. Moderate improvements are expected for Q1 and Q5, where telemetry access is present but is either partially supported by indexes or used mainly inside subqueries. Overall, the analysis suggests that columnar storage will primarily improve telemetry-intensive and aggregation-heavy workload operations.

Propose usage for in-memory expressions and join groups

1. In-Memory Expressions

Expression candidate:

EXTRACT(YEAR FROM activity_date)

Type	Transaction
Queries	Q1,Q2,Q3
Crud	Q6

In-Memory Expressions are proposed for frequently repeated filter expressions, especially EXTRACT(YEAR FROM activity_date), which appears in several workload transactions. Precomputing this expression may reduce repeated calculation costs during query execution and improve filtering efficiency. This feature is expected to affect analytical SELECT transactions such as **Q1**, **Q2** and **Q3**, as well as the DELETE transaction **Q6**, which also filters activities by year.

2. Join Groups

Join Groups candidate:

ACTIVITIES.runner_id = RUNNERS.runner_id

ACTIVITIES.route_id = ROUTES.route_id

Type	Transaction
Queries	Q1,Q2,Q3
Crud	Q5,Q6

Based on the query plan analysis, the strongest Join Group candidates are the joins between ACTIVITIES and RUNNERS using runner_id, and between ACTIVITIES and ROUTES using route_id. These joins appear repeatedly in several workload transactions and are visible in the query plans through HASH JOIN and NESTED LOOPS operations. They are used by both analytical SELECT transactions and CRUD transactions, especially Q2, Q3 and Q6.

Experiments

I) INMEMORY on whole table vs INMEMORY on selected columns

Selected columns:

activity_id, heart_rate_bpm, pace, running_power

This experiment compares two Oracle In-Memory storage approaches: storing the entire TELEMETRY_DATA table in columnar format versus storing only selected frequently used columns. The purpose of this comparison is to determine whether limiting In-Memory storage to workload-critical columns can provide similar or better performance than storing the whole table. Since the query plans show that only selected telemetry columns are repeatedly used for joins and analytical calculations, this experiment helps evaluate the balance between query performance and memory efficiency. This comparison may also indicate whether storing unnecessary columns in memory introduces additional memory overhead without proportional performance gains.

II) MEMCOMPRESS FOR QUERY LOW vs MEMCOMPRESS FOR QUERY HIGH on TELEMETRY_DATA.

This experiment compares two Oracle In-Memory compression methods applied to the TELEMETRY_DATA table: MEMCOMPRESS FOR QUERY LOW and MEMCOMPRESS FOR QUERY HIGH. The purpose is to evaluate whether lower compression, which may provide faster access, or higher compression, which may reduce memory usage, gives better performance for telemetry-heavy workload transactions. The experiment will focus on Q2, Q3, Q4 and Q6 because these transactions repeatedly scan and aggregate data from TELEMETRY_DATA.